

```
In [ ]: import marimo as mo
```

Universidad de Costa Rica | Escuela de Ingeniería Eléctrica

IE0405 - Modelos Probabilísticos de Señales y Sistemas

PyX - Serie de tutoriales de Python para el análisis de datos

Py1 - *Funciones y librerías estándar*

Dentro de las librerías estándar de Python hay herramientas útiles para operaciones numéricas básicas, para el manejo de archivos como los que se van a aplicar en el curso y otras funciones útiles para programación en general.

Fabián Abarca Calderón

Funciones, librerías y métodos

En programación generalmente se acepta que "no hay que reinventar la rueda" y también que "no hay que repetirse", y además que "hay que mantenerlo sencillo". Para no olvidarlo, existen estos acrónimos:

- **DRY**, *don't repeat yourself*
- **KISS**, *keep it simple, stupid*

Una forma de seguir estos buenos consejos es creando funciones que se invocan cuando hay que hacer tareas repetitivas, y también utilizando piezas de código ("librerías") que ya han sido desarrolladas para resolver aplicaciones específicas.

La existencia de funciones y librerías (con sus módulos y métodos asociados), desarrollados por una inmensa comunidad global, agregan muchas funcionalidades poderosas a Python.

1.1 - Funciones

En Python, la sintaxis para la creación de una función es:

```
def nombre():  
    <acción de la función>
```

```
In [ ]: def suma(x, y):  
        z = x + y  
        return z  
  
print(suma(1, 2))
```

1.2 - Librerías, módulos, métodos y atributos

Las librerías son grupos de funciones predefinidas, que se pueden importar al código en conjunto y utilizarlas para realizar tareas de forma más sencilla, reduciendo así el tamaño del código propio y simplificándolo.

Para llamarlas se utiliza

```
import <libreria>
```

donde también se utiliza un "alias": un nombre corto (muy corto, como `np` para `numpy`) para referirse a este por el resto del programa.

```
import <libreria> as <alias>
```

Sin embargo en ocasiones no se desea importar toda la librería sino solo algunos componentes o "módulos":

```
from <libreria> import <módulo o método>
```

Luego de importarlas, se invocan sus atributos y métodos con la notación del punto. Por ejemplo, la función coseno de la librería NumPy es `np.cos()` (un método) y el número π es `np.pi` (un atributo). Si se importa un método directamente, no es necesaria la notación del punto, por ejemplo: `randint()` (una función de `random`).

```
In [ ]: import math  
import numpy as np  
from random import randint  
from scipy.constants import c  
  
print('4! =', math.factorial(4))  
print('C/D =', np.pi)  
print('Un número aleatorio =', randint(50, 60))  
print('Velocidad de la luz =', c)
```

1.2.1 - Librería `math`

Estas son funciones matemáticas comunes. Hay una gran [variedad](#), entre las que se incluyen:

Redondeo

- `math.ceil(x)` : retorna el "techo" de `x`, el entero más pequeño mayor o igual que `x`.
- `math.floor(x)` : retorna el "piso" de `x`, el entero más grande menor o igual que `x`.

Análisis combinatorio

- `math.comb(n, k)` : retorna el coeficiente binomial $C(n, k)$, correspondiente al número de combinaciones de `k` elementos que se pueden obtener de un grupo de `n` elementos, donde el **orden no importa**.

$$\binom{n}{k}$$

- `math.perm(n, k)` : retorna el número de formas de elegir `k` elementos de `n` elementos sin repetición, donde el **orden sí importa**.

Otras funciones útiles

- `math.factorial(x)` : retorna el factorial de `x` : $x!$.
- `math.pow(x, y)` : retorna el valor de x^y .
- `math.sqrt(x)` : retorna la raíz cuadrada de `x` : $\sqrt{x}$.
- `math.erf(x)` : retorna el valor de la función de error (la función de distribución normal estándar) evaluada en `x`. Esta función será de gran utilidad en este curso.

```
In [ ]: print(math.ceil(2.3))
        print(math.floor(2.3))
        print(math.erf(1.1))
```

1.2.2 - Librería `random`

Generación de números pseudo-aleatorios, con distintas distribuciones. Hay una gran [variedad](#), entre las que se incluyen:

- `random.randint(a, b)` : retorna un número aleatorio entero entre `a` y `b` (intervalo cerrado).
- `random.uniform(a, b)` : retorna un número aleatorio flotante para una distribución uniforme entre `a` y `b`.
- `random.sample(population, k)` : retorna una lista con `k` muestras aleatorias tomadas de la lista `population`.
- `random.expovariate(lambd)` : retorna un número aleatorio flotante para una distribución exponencial con parámetro `lambd`.
- `random.gauss(mu, sigma)` : retorna un número aleatorio (flotante) para una distribución normal con media `mu` y desviación estándar `sigma`.

```
In [ ]: import random as rd
print(rd.gauss(1,3))
print(rd.randint(0,100))
print(rd.uniform(0,100))
```

1.2.3 - Librería `statistics`

Herramientas estadísticas para aplicar a un conjunto de datos. Hay una gran [variedad](#), entre las que se incluyen:

- `statistics.mean(data)` : retorna el valor esperado de un conjunto de datos `data` .
- `statistics.pstdev(data)` : retorna la desviación estándar de la población para el conjunto de datos `data` .
- `statistics.stdev(data)` : retorna la desviación estándar de una muestra para el conjunto de datos `data` .
- `statistics.pvariance(data)` : retorna la varianza de la población para el conjunto de datos `data` .
- `statistics.variance(data)` : retorna la varianza de una muestra para el conjunto de datos `data` .

Entre otros. Consultar la documentación adjunta.

```
In [ ]: import statistics

data = range(1, 60)

print(statistics.mean(data))
print(statistics.variance(data))
```

1.2.4 - Librería `collections`

Presenta alternativas a las listas, sets, tuplas y diccionarios que incluye Python por defecto. Incluye diferentes estructuras para almacenar y manejar datos, definidas por clases, cada una con sus métodos específicos Hay una gran [variedad](#), entre las que se incluyen:

- `collections.deque` : Agrega funcionalidades a una lista convencional, como pops y appends.
- `collections.OrderedDict` : Diccionario que registra el orden en que fueron agregados los objetos.
- `collections.UserString` : Agrega funcionalidades para manejo de objetos tipo `String` .

El siguiente es un ejemplo para la solución "elegante" de un problema común.

¿Cuál es la probabilidad de que, al lanzar dos dados, la suma de los dados sea 7? [2]

(El resultado es fácil de deducir: de 36 combinaciones posibles, seis suman siete (1 + 6, 2 + 5, 3 + 4, 4 + 3, 5 + 2, 6 + 1), entonces $6/36 = 1/6 \approx 0.16667$).

Primero, el objeto `defaultdict` del módulo `collections` crea diccionarios con valores predeterminados cuando encuentra una nueva clave. Su uso práctico es el de **"diccionario rellenable"**.

```
In [ ]: from collections import defaultdict
```

Ahora, es posible crear un diccionario con todas las combinaciones posibles y la suma de cada una, con un doble bucle `for` :

```
In [ ]: d = {(i,j) : i+j for i in range(1, 7) for j in range (1, 7)}  
print(d)
```

Seguidamente se crea un `defaultdict` vacío. Este implica que, más adelante, si una clave no es encontrada en el diccionario, en lugar de un `KeyError` se crea una nueva entrada (un nuevo `key:value`).

```
In [ ]: dinv = defaultdict(list)  
print(dinv)
```

Es posible extraer del diccionario las combinaciones que suman 7. El método `.items()` genera una lista de pares de "tuplas" (una tupla es un conjunto ordenado e inmutable de elementos) a partir del diccionario de combinaciones creado en `d` .

"Rellenamos" el `defaultdict` con los elementos en el diccionario creado anteriormente y el método `.append()` , esto con un bucle `for` en donde los índices `i,j` representan los pares de combinaciones y su suma. La ventaja es que ahora están todos agrupados.

```
In [ ]: print('Antes...\n')  
print(d.items())  
  
for i,j in d.items():  
    dinv[j].append(i)  
  
print('\nDespués...\n')  
dinv
```

El `for` anterior puede leerse como: "para cada par en la lista de ítemes, en la posición `j` (la suma de las combinaciones) añada la combinación correspondiente (en `i`)".

Extraemos los pares que suman siete y obtenemos la cantidad de estos.

```
In [ ]: print('Combinaciones que suman 7:', dinv[7])
        print('Elementos:', len(dinv[7]))
```

Finalmente, y más en general, se obtiene la probabilidad para todas las sumas en forma de un solo diccionario:

```
In [ ]: probabilidades = {i : len(j)/36 for i,j in dinv.items()}
        print('El vector de probabilidades de suma es =', probabilidades)
        print('La probabilidad de que la suma sea 7 es =', probabilidades[7])
```

1.2.5 - Librería csv

Esta librería implementa clases para el manejo de archivos tipo CSV (*Comma Separated Values*), con actividades típicas como *leer* datos desde y *escribir* datos en un archivo CSV u otros similares.

Entre sus funciones se encuentran:

- `csv.reader(csvfile, dialect='excel', **fmtparams)` : crea un objeto tipo `reader` con los datos del archivo `csvfile` para el "dialecto" (formato) especificado. `fmtparams` son "parámetros de formato" adicionales para modificar la configuración del formato.
- `csv.writer(csvfile, dialect='excel', **fmtparams)` : crea un objeto tipo `writer` para escribir datos al archivo `csvfile` con el "dialecto" (formato) especificado. `fmtparams` son "parámetros de formato" adicionales para modificar la configuración del formato.

La [documentación](#) completa tiene todos los detalles.

```
In [ ]: import csv

unos = [1]*10

with open('unos.csv', 'w', newline='') as archivo:
    escribir = csv.writer(archivo)
    escribir.writerow(unos)
```

1.2.6 - Librería os

Permite manejar archivos y rutas del sistema operativo como si fueran comandos en terminal. La lista completa de funciones se presenta en la [documentación](#). Los métodos más importantes son:

- `os.getcwd()` : retorna el directorio de trabajo actual.
- `os.chdir(path)` : Cambia el directorio de trabajo al especificado por `path`.
- `os.path` es un módulo para manipulación de direcciones (rutas) del sistema.

```
In [ ]: import os

print(os.getcwd())
```

1.2.7 - Otras librerías

1.2.7.1 - Librería `datetime`

Presenta clases para manipulación de fechas y tiempo. Tiene dos módulos: `calendar` y `time`, cada uno con sus funciones, y permiten generar información como la fecha y hora actuales, la zona horaria, entre otros. La documentación completa se encuentra [aquí](#) y excelentes ejemplos en [Programiz](#).

```
In [ ]: import datetime
_ahora = datetime.datetime.now()
print(_ahora)
print(_ahora.month)
```

1.2.7.2 - Módulo `calendar`

Presenta una clase `Calendar` que permite crear objetos que representen calendarios, e incluye métodos para manipularlos. La documentación completa se encuentra [aquí](#).

```
In [ ]: import calendar
_ahora = datetime.datetime.now()
es_bisiesto = calendar.isleap(_ahora.year)
calendario = calendar.month(_ahora.year, _ahora.month)
print('Año bisiesto:', es_bisiesto)
print(calendario)
```

1.3 - ¿Cómo crear una librería propia?

Una librería consiste simplemente en una serie de archivos de código (extensión `.py`) con definiciones de las funciones a utilizar. Luego de importar la librería a un código, estas funciones se pueden acceder y utilizar. Los archivos de la librería se pueden distribuir de una forma jerárquica, teniendo de esta forma "sublibrerías" y permitiendo así clasificar las funciones.

Para crear un paquete se requieren dos elementos:

- Todos los archivos `.py` con las funciones en una única carpeta, con el nombre de la librería
- Un archivo `__init__.py` (que normalmente se deja vacío) para que el intérprete identifique la carpeta como una librería

Hay muchos [recursos](#) en línea que explican y ejemplifican este procedimiento.

Más información

- [Documentación oficial de Python](#)
-

Universidad de Costa Rica | Facultad de Ingeniería | Escuela de Ingeniería Eléctrica

© 2021
